Labeling Instructions for Code Review Comments

This document provides step-by-step instructions for labeling code review comments with **Quality**, **Severity**, **Context Scope**, and **Advanced**,. Comments may come from either human reviewers or AI code review — the labeling criteria are the same regardless of source. The goal is to build a high-quality evaluation dataset that includes the best comments from both humans and AI.

Step-by-Step Labeling Workflow

Step 0: Open the PR url

- Read the PR title and description to understand what the repo is about and what the PR changes

Step 1: Go the the discussion where comment was made

- Click on **discussion_url** which takes you directly to the review comment, shown alongside its surrounding diff hunk.
- Double check if the comment matches with the "**body**" field in the input data.

Step 2: Review on diff and changed file level

- From the discussion view, click on the **file name** at the top of the diff, which takes you to the PR's Files Changed tab.
- The **diff_line** value from the input data will help you locate the specific line number of the file where the comment is anchored.
- **Read through the full diff hunks across multiple changed files** to build a comprehensive understanding of this PR.

Step 3: Browse the repository

- If the comment requires context beyond the changed files, use **repo_url** to browse the repo tree at the PR head commit.
- Explore the files in the repository at the correct snapshot, look up imports, shared utilities, base classes, or any other file mentioned or implied by the comments at the repo level.

Step 4: Read and label the review comment

- Read the "body" field in the input data again. This will be the review comment you need to label.
- To fully understand what issue this comment is pointing out and specific code context needed to make such a comment, you may need to explore actively using the navigation in Step 1-3 repetitively.

Step 5: Label the 4 Axes

With the diff, comment, and any needed context in hand, assign labels for each axis:

1. **Quality**: Is the comment helpful, unhelpful, or wrong?
2. **Severity**: Is the issue nit, moderate, or critical?
3. **Context Scope**: What level of context is needed (diff, file, repo, or external) to produce such a comment, and where is the context located?
4. **Advanced**: Is this an advanced issue that is particularly valuable for a benchmark dataset (true/false)?

Record your labels in the JSON format described in the **Labeling Format** section below.

Axis 1: Quality

Classify the overall quality of the review comment. Choose exactly one quality label from below.

Label	Definition	Example
helpful	Comment identifies a genuine issue, suggests a meaningful improvement, or catches a real bug.	"This null check should happen before the dereference on L42, not after."
unhelpful	Comment is pedantic, stylistic without substance, obvious, or not actionable.	"Consider adding a comment here." (on self-documenting code)
wrong	Comment is factually incorrect, suggests a change that would introduce a bug, or misunderstands the code.	"This will cause an integer overflow" (when the type is actually a long).

Axis 2: Severity

Assess how severe the issue is that the review comment points out. Choose exactly one of the three levels below.

Label	Definition	Example
nit	Minor issues that can safely be ignored or deferred. They pose no meaningful risk and do not affect the code's behavior.	Style nit-picks, naming preferences, cosmetic suggestions, readability improvements, documentation gaps, minor code smell.
moderate	Issues that affect or could affect the code's behavior but are unlikely to cause serious harm. The code works but should be improved.	Missing edge case handling that is unlikely to occur, suboptimal but functional logic, missing null checks on uncommon paths.
critical	Severe issues that pose a significant and concrete risk to the code's correctness, security, or stability. A senior engineer would insist this be fixed before merging.	SQL injection vulnerability, authentication bypass, data corruption, race conditions in concurrent code, returning wrong results, build-breaking errors.

Severity Labeling Rules:

1. Focus on the issue itself, not the comment's wording. Even a well-written comment about a trivial style is still "nit."
2. If the comment points out multiple issues, rate by the most severe issue mentioned.
3. If you are unsure between two levels, consider: "Would a senior engineer insist this be fixed before merging?" If yes, lean toward “critical”.

Axis 3: Context Scope

Determine what level of context a reviewer would need to confidently make this comment. This tells us how much surrounding information was required to identify the issue. Choose exactly one.

Label	Definition	Example
diff	The comment can be made by reading **only the PR's changed lines (the diff hunks)** — possibly across multiple changed files in the same PR. No context beyond the diff is needed.	Typos in new code, obvious syntax errors, clearly wrong constants, style violations visible in the diff alone, inconsistency between two diff hunks in different files of the same PR.
file	The comment requires reading **beyond the diff hunks but within the file(s) that the PR touches**. The reviewer needed surrounding functions, imports, class definitions, or other code that is in a PR-changed file but outside its changed lines. A PR can touch multiple files, so "file" scope can span several of them.	Using a variable defined elsewhere in the file, contradicting a pattern established earlier in the same file, missing an import that exists at the top of the file, duplicating logic already present in a different file that the PR also modifies.
repo	The comment requires knowledge from **files not changed by the PR** — other source files, configuration files, tests, documentation, or build scripts elsewhere in the repository.	Inconsistency with a shared utility function in an untouched file, violating a pattern set in a base class the PR doesn't modify, missing an update to a config file not in the PR, breaking an API contract defined in another module.
external	The comment requires knowledge **outside the repository entirely** — such as business requirements, customer expectations, or domain-specific constraints that are not documented in the repo.	The product team decided this feature must support offline mode but the implementation assumes constant connectivity.

How to Record Context Scope

Two fields work together:

1. **`context_scope`** — a single string (`"diff"`, `"file"`, `"repo"`, or `"external"`) that captures the **broadest** level of context the reviewer needed.

2. **`context`** — an array of objects listing every specific piece of evidence the reviewer used. Each entry has three fields:

    1. diff_line:
        - A line number or range (e.g., "42", "42-50") in the file where the relevant code lives.

    - Set to null if it's too hard to locate the exact diff hunk, or the entire file / a broad section is relevant.
2. file_path
    - Repo-relative path to the file (e.g., src/utils/http_client.py)
3. why:
    - A short phrase explaining why this piece of context matters to the review comment — i.e., what understanding it provides that supports (or would be needed to evaluate) the reviewer's claim.

Context Labeling Rules:

- **`context_scope`** should reflect the broadest scope needed. If the reviewer needed both diff and file context, set it to `"file"`. If they also needed a repo file not touched by the PR, set it to `"repo"`.
- List **all** pieces of context in the `context` array. If only the diff was needed, one entry is enough. If the reviewer also needs to read another part of the file or another file, add those as additional entries.
- If `context_scope` is `"external"`, the `context` array may be **empty** (`[]`) since the knowledge comes from outside the repository (e.g., language docs, framework behavior, RFCs).

Axis 4: Advanced

Indicate whether the review comment goes beyond what is obvious from reading the changed lines alone. A comment is advanced if it requires deeper knowledge or reasoning that most reviewers would not have just from looking at the diff. This is a binary label.

Label it as "true" if it meets one or more of the following criteria:

Criterion	Description
Repo-specific conventions	The comment pertains to conventions, patterns, or architectural decisions specific to this repository that are not universally known.
Context outside changed files	The comment requires knowledge from files not touched by the PR (e.g., base classes, shared utilities, config, or API contracts defined elsewhere in the repo).
Recent language/library updates	The comment requires awareness of recent or non-obvious language features, library behavior, deprecations, or framework-specific semantics.
Better implementation approach	The comment suggests a meaningfully better way to implement the code — not just style, but a fundamentally improved design, algorithm, or API usage.

Label it as "false" if it doesn't meet any of the criteria.

Labeling Format

Examples

Input data:


```
{
  "pull_request_url": "https://github.com/ai-code-review-evaluation/sentry-greptile/pull/8",
  "nwo": "ai-code-review-evaluation/sentry-greptile",
  "head_sha": "c3b32a7babfcf62b91553388bcd9e9fbfd30a690",
  "comment_id": "example-003",
  "body": "Line 162 copies the config and transforms `schedule_type` in the copy, but line 168
passes `monitor_environment.monitor.config` (the original, untransformed config) into the
return dict. This defeats the purpose of the transformation — the `config` local variable
should be used instead.",
  "file_path": "src/sentry/monitors/logic/incident_occurrence.py",
  "diff_line": 168,
  "discussion_url": "https://github.com/ai-code-review-evaluation/sentry-
greptile/pull/8#discussion_rexample003",
  "repo_url": "https://github.com/ai-code-review-evaluation/sentry-
greptile/tree/c3b32a7babfcf62b91553388bcd9e9fbfd30a690"
}
```


PR Title: Reorganize incident creation / issue occurrence logic

Diff (in src/sentry/monitors/logic/incident_occurrence.py — new file, showing lines 160–171):

```
+def get_monitor_environment_context(monitor_environment: MonitorEnvironment):
+    config = monitor_environment.monitor.config.copy()
+    if "schedule_type" in config:
+        config["schedule_type"] = monitor_environment.monitor.get_schedule_type_display()
+
+    return {
+        "id": str(monitor_environment.monitor.guid),
+        "slug": str(monitor_environment.monitor.slug),
+        "name": monitor_environment.monitor.name,
+        "config": monitor_environment.monitor.config,
+        "status": monitor_environment.get_status_display(),
+        "type": monitor_environment.monitor.get_type_display(),
+    }
```

Labels:

```
{
  "quality": "helpful",
  "severity": "critical",
  "context_scope": "file",
  "context": [
    {
      "diff_line": "168",
      "file_path": "src/sentry/monitors/logic/incident_occurrence.py",
      "why": "return dict passes the original untransformed config instead of the local
`config` copy — the schedule_type transformation is silently lost"
    },
    {
      "diff_line": "161-163",
      "file_path": "src/sentry/monitors/logic/incident_occurrence.py",
      "why": "`config` is copied and schedule_type is transformed here — this is the variable
the return dict should reference"
    }
  ],
  "advanced": false
}
```

Reasoning:

1. **Quality = helpful**: The comment correctly identifies that the code copies and transforms config but then ignores the copy, passing the original untransformed config in the return dict. The fix is clear: replace monitor_environment.monitor.config with config on line 168.
2. **Severity = critical**: The schedule_type display transformation is silently lost. Downstream consumers (e.g., issue evidence displays for monitor incidents) will receive the raw schedule type integer instead of a human-readable string, producing confusing or broken output.
3. **Scope**: The bug line (168) is in the diff, but understanding why it is wrong requires reading lines 161–163 of the same file, where config is created and transformed. The file is new in this PR, so the reviewer needed to read non-diff-hunk lines within the same file — scope is "file".
4. **Advanced = false**: Noticing that a local variable is created but not used in the return dict is a common code review pattern.

Frequently Asked Questions

Q1: How should better distinguish between quality and severity?
Quality measures whether the comment is correct and useful, and Severity measures how serious the underlying issue is. A comment can be perfectly correct and useful (helpful) while pointing out a minor style preference (nit). Similarly, a comment can be Wrong even if the issue it tried to flag would have been critical. Always evaluate the two axes independently.

Q2: If `context_scope` is "diff", does that mean only the diff hunk where the comment was placed, and therefore the context needed would just be the diff hunk being commented on?
Not. Set context_scope as “diff” means the reviewer only needed the PR's changed lines, but a PR can have changed lines across multiple files. If the reviewer needs context from two diff hunks in two different files, the scope is still “diff” because no context beyond the changed lines was needed, and the “context” array should list entries from both files.

Q3: If I label the “context_scope” field as “file”, where would I still need to fill in the “diff_line” under the “context” field?

If context_scope = "file": You still fill in diff_line for any context entry that points to a specific line, including lines that are inside the diff. Look at the example section where the context_scope is "file" but both context entries have diff_line values ("168" and "161-163"), because those are the specific lines the reviewer needed to look at, even though the scope is "file" (the reviewer needed to read beyond the diff hunks within the same file).